

Number: P3367R1

Date: 2024-11-22

Audience: CWG

Reply-to: [Hana.Dusíková](#)

Table of contents

- [Changes](#)
- [Why would you want to do this?](#)
- [Simple example](#)
- [Implementation experience](#)
- [Impact on existing code](#)
- [Intention for wording changes](#)
- [Proposed changes to wording](#)
 - [Feature test macros](#)

constexpr coroutines

This paper is proposing making coroutines functional during constant evaluation. Even when most of use-cases for coroutines are based on I/O and event based, coroutines are still useful for compile-time based computation, eg. `std::generator`.

Changes

- [R0](#) → [R1](#): Changed wording mentioning lifetime of coroutines after `expr.const` 5.2

Why would you want to do this?

Well, you just told me coroutines are the best way to solve some problems, so wouldn't I also want to use the Best Way at compile time? (quote from Jason Turner, co-author of *"constexpr all the things"* talk)

Simple example

When this paper is merged into the standard, users will be able to use `std::generator`-like coroutines to generate or calculate data.

```
template <typename T> constexpr auto fib
{
    T a = 0;
    T b = 1;
    co_yield a;
    do {
        co_yield b;
        auto tmp = b;
        b += a;
        a = tmp;
    } while (true);
}

template <typename T, size_t N> constexpr
auto res = std::array<T, N>{};
std::ranges::copy(fib<T>() | std::views::take(N), res);
return res;
}
```

```
constexpr auto cached_fibonacci = calculate_fibonacci;
```